Scoring financier sur données chiffrées via FHE : prototype et limites

Résumé personnel du cours — BROUILLON DE TRAVAIL v0, à réécrire par les auteurs

Ravin THILAGARASA

Thomas GRUNDSZTEJN

ESGI Paris · FYC 2026-2027

> **Brouillon de travail, pas un rendu.** Ce texte sert de canevas : la structure, les chiffres et les références sont vérifiés, mais la rédaction finale doit être entièrement réécrite par nous avant le dépôt sur Compilatio (règlement FYC : la génération automatique est sanctionnée). Les passages marqués **[À COMPLÉTER]** attendent notre expérience personnelle ; les références marquées **[à vérifier]** doivent être confirmées avant d'être citées.

1. Introduction : le maillon manquant de la confidentialité

Quand une banque ou une fintech décide d'accorder un crédit, elle calcule un score à partir de données très personnelles : le revenu, le taux d'endettement, l'historique des incidents de paiement, parfois le comportement de consommation. Ces données sont aujourd'hui bien protégées quand elles dorment sur un disque (chiffrement au repos) ou quand elles circulent sur le réseau (TLS). Mais au moment précis où le score est calculé, elles doivent être déchiffrées et chargées en mémoire. C'est le maillon manquant de la chaîne : un hébergeur cloud, un administrateur malveillant, un prestataire de scoring ou une simple fuite mémoire suffit alors à les exposer.

Le Règlement général sur la protection des données rend ce point très concret. L'article 5 impose la minimisation des données et l'article 32 la sécurité du traitement : moins un acteur voit de données en clair, mieux le principe est respecté. Dans le secteur bancaire, la question se double d'un enjeu de souveraineté dès que le calcul est confié à un service hébergé hors de l'Union européenne.

Le chiffrement totalement homomorphe (FHE, *Fully Homomorphic Encryption*) propose de fermer ce dernier maillon : il permet d'effectuer des calculs directement sur des données chiffrées, et de n'obtenir le résultat en clair qu'au moment où son destinataire légitime le déchiffre. Appliqué au scoring, cela signifie qu'un serveur peut produire un score de crédit qu'il est lui-même incapable de lire.

Notre travail part donc d'une question simple : **peut-on calculer un score de crédit sur des données qui restent chiffrées de bout en bout, et est-ce exploitable aujourd'hui dans une banque ou une fintech ?** Pour y répondre, nous avons construit un cours complet sur Moodle, un prototype qui compare trois façons de chiffrer le même calcul, et un atelier d'exercices en ligne de commande, `fhe-cli`, publié sur PyPI. Ce résumé reprend le fil du cours, présente ce que le prototype a réellement mesuré, puis porte un regard critique sur la maturité de la technologie.

[À COMPLÉTER] Pourquoi nous avons choisi ce sujet, ce que nous en savions au départ et ce qui nous a surpris en avançant.

2. Du chiffrement classique au chiffrement homomorphe

2.1 Ce qu'un chiffrement classique ne permet pas

Un chiffrement symétrique moderne comme AES, utilisé par exemple à travers Fernet en Python, est conçu pour que le chiffré ne révèle rien du message. Cette propriété a une contrepartie : on ne peut rien calculer d'utile sur le chiffré. Additionner deux chiffrés AES octet par octet ne donne pas le chiffré de la somme, mais une suite d'octets sans signification. Pire, Fernet ajoute un code d'authentification : la moindre modification du chiffré est détectée et le déchiffrement est refusé. C'est exactement ce que l'on attend pour stocker ou transporter des données, et exactement ce qui empêche de les traiter.

Les premiers exercices de notre atelier font toucher du doigt ces deux extrêmes. Le chiffre de César et le XOR montrent qu'un chiffrement trop simple est *malléable* : on peut modifier le message sans le connaître, ce qui est une faiblesse. Fernet montre l'inverse : aucune manipulation n'est possible. Le chiffrement homomorphe se situe entre les deux : il rend certaines manipulations possibles, mais de façon contrôlée et utile.

2.2 Le chiffrement partiellement homomorphe : Paillier

Le cryptosystème de Paillier (1999) est l'exemple le plus parlant d'un chiffrement *partiellement* homomorphe (PHE). Il repose sur la difficulté de factoriser un grand nombre $n = p \cdot q$, produit de deux nombres premiers secrets. Un message m est chiffré en $c = g^m \cdot r^n \bmod n^2$, où g vaut classiquement $n + 1$ et r est un aléa tiré à chaque chiffrement. Deux chiffrements du même message sont donc différents, ce qui empêche de reconnaître une valeur déjà vue.

La propriété remarquable est la suivante : multiplier deux chiffrés donne le chiffré de la somme des messages, et élever un chiffré à une puissance k donne le chiffré de k fois le message. Autrement dit, sans jamais connaître les valeurs, un serveur peut additionner des montants chiffrés et les multiplier par des coefficients qu'il connaît en clair. Pour un score linéaire dont les poids sont publics, c'est suffisant. En revanche, Paillier ne sait pas multiplier deux valeurs chiffrées entre elles, ni comparer un résultat à un seuil.

Un exemple jouet, avec des nombres bien trop petits pour être sûrs, permet de le vérifier à la main. Prenons $p = 5$ et $q = 7$, donc $n = 35$, $n^2 = 1\,225$ et $g = 36$. Le chiffrement de 3 avec l'aléa $r = 2$ donne 683 ; celui de 4 avec $r = 3$ donne 1 062. Le serveur, qui ne voit que 683 et 1 062, calcule leur produit modulo 1 225 et obtient 146. Le détenteur de la clé déchiffre 146 et trouve 7, soit 3 + 4. De même, 683 élevé à la puissance 5 se déchiffre en 15, soit 5 × 3. Et si l'on chiffre de nouveau 3 avec un autre aléa, $r = 4$, on obtient 44 : rien dans les chiffrés ne trahit que 683 et 44 cachent la même valeur. Dans la réalité, n fait au moins 2 048 bits, et chaque chiffré occupe donc 4 096 bits.

Dans l'atelier, les exercices 4 à 9 reconstruisent Paillier pas à pas : exponentiation modulaire, inverse modulaire, génération des clés, chiffrement et déchiffrement, propriétés homomorphes, puis calcul d'un score chiffré. Implémenter ces briques à la main nous a semblé la meilleure façon de démystifier l'idée de « calculer sans voir ».

2.3 Du partiel au total : LWE, le bruit et le bootstrapping

Un chiffrement est dit *totalement* homomorphe s'il permet à la fois l'addition et la multiplication entre chiffrés, un nombre arbitraire de fois. Avec ces deux opérations, on peut en principe évaluer n'importe quel circuit, donc n'importe quel calcul. Le problème est resté ouvert trente ans, jusqu'à la thèse de Craig Gentry en 2009.

Les schémas modernes reposent sur le problème *Learning With Errors* (LWE), introduit par Regev en 2005, et sur sa variante algébrique Ring-LWE (Lyubashevsky, Peikert et Regev, 2010), plus compacte et plus rapide. L'idée est de cacher le message dans une équation linéaire volontairement

perturbée par un petit *bruit* aléatoire. Retrouver le secret à partir d'équations bruitées est un problème difficile, y compris, selon l'état actuel des connaissances, pour un ordinateur quantique : c'est la même famille de problèmes que celle retenue par le NIST pour la cryptographie post-quantique.

L'intuition peut se donner sans démonstration. Sans bruit, un système d'équations linéaires dont le secret est l'inconnue se résout en quelques lignes par la méthode du pivot de Gauss. Il suffit d'ajouter à chaque équation une petite erreur aléatoire, de l'ordre de quelques unités alors que les coefficients sont gigantesques, pour que le pivot amplifie ces erreurs au point de rendre le résultat inutilisable. Le détenteur du secret, lui, sait retirer la partie « équation » et n'a plus devant lui qu'un message entaché d'un petit bruit, qu'il élimine par un simple arrondi. Tout l'art des schémas FHE consiste à faire en sorte que les opérations sur les chiffrés se traduisent par les mêmes opérations sur les messages, tout en gardant le bruit sous ce seuil d'arrondi.

Ce bruit est à la fois la source de la sécurité et la principale contrainte pratique. Chaque opération homomorphe l'augmente : une addition l'additionne, une multiplication le multiplie, ce qui est beaucoup plus brutal. Au-delà d'un certain seuil, le bruit déborde sur le message et le déchiffrement devient faux. Un schéma qui supporte seulement un nombre limité d'opérations est dit *somewhat* ou *leveled* : on choisit à l'avance une profondeur multiplicative, et les paramètres en découlent.

L'apport décisif de Gentry est le *bootstrapping* : évaluer homomorphiquement l'algorithme de déchiffrement lui-même, à l'aide d'une clé secrète chiffrée, pour obtenir un nouveau chiffré du même message avec un bruit remis à un niveau bas. Le bootstrapping rend le nombre d'opérations illimité, mais c'est aussi l'opération la plus coûteuse de toute la chaîne. Une grande partie de la recherche des quinze dernières années vise à le rendre plus rapide, ou à s'en passer en dimensionnant correctement les paramètres.

2.4 Quatre familles de schémas

Le cours présente quatre familles, que nous avons classées selon leur intérêt pour le scoring :

Schéma	Opérations	Données	Usage pour le scoring
Paillier (1999, PHE)	addition entre chiffrés, produit par un clair	entiers	score linéaire à poids publics
BGV / BFV (2012)	addition et multiplication, profondeur limitée	entiers exacts	règles métier, comptages exacts
CKKS (2017)	addition et multiplication approchées, vecteurs	réels	régression, réseaux de neurones
TFHE (2016-2020)	portes logiques, tables de correspondance	bits, petits entiers	comparaisons, seuils de décision

BGV et BFV travaillent sur des entiers modulo un nombre t et donnent des résultats exacts. CKKS, proposé par Cheon, Kim, Kim et Song en 2017, accepte de traiter le bruit comme une petite erreur d'arrondi : les résultats sont approchés, mais il manipule directement des nombres réels et peut regrouper des milliers de valeurs dans un seul chiffré (technique SIMD), ce qui en fait le schéma de référence pour l'apprentissage automatique. TFHE, enfin, travaille bit par bit avec un bootstrapping très rapide, de l'ordre de la dizaine de millisecondes ; il est le plus adapté aux opérations non linéaires comme une comparaison à un seuil, et c'est le schéma qu'utilise Zama.

CKKS mérite un mot de plus, car c'est lui qui fait tourner la partie « vrai FHE » de notre prototype. Un nombre réel y est multiplié par un grand facteur d'échelle, 2^{40} dans notre démo, puis arrondi en entier avant d'être chiffré. Après une multiplication, l'échelle est élevée au carré : une opération de *rescaling* la ramène à sa valeur initiale en consommant un des modules de la chaîne de paramètres. Le nombre de modules disponibles fixe donc directement la profondeur multiplicative du calcul.

Choisir ces paramètres revient à arbitrer entre profondeur, précision, sécurité et taille des chiffrés : c'est le principal savoir-faire que demande la technologie.

[À COMPLÉTER] La notion qui nous a demandé le plus d'effort à comprendre (bruit, niveaux, mise à l'échelle CKKS…) et la façon dont nous avons choisi de l'expliquer dans le cours.

3. Le scoring financier : ce qu'il faut calculer

Un score de crédit résume le risque qu'un emprunteur ne rembourse pas. La forme la plus simple est un score linéaire pondéré : chaque indicateur est multiplié par un poids, puis les termes sont additionnés à un biais. La régression logistique, très utilisée dans les banques pour sa lisibilité réglementaire, ajoute une étape : la somme pondérée passe dans une fonction sigmoïde qui la transforme en probabilité de défaut, comprise entre 0 et 1.

Notre prototype utilise trois indicateurs : le revenu mensuel net R (en euros), le taux d'endettement E (en pourcentage) et le nombre I d'incidents de paiement sur douze mois. Le score vaut :

$$S = 50 + 0{,}005 \times (R - 2\,500) - 0{,}9 \times E - 10 \times I$$

Les poids sont publics : ils appartiennent au modèle de la banque, pas au client. Seules les données du client sont secrètes. Le revenu est centré autour de 2 500 € pour que le biais garde un sens : un client « moyen », sans dette ni incident, obtient 50. Ces coefficients sont volontairement simples et pédagogiques ; un modèle réel serait appris sur un historique de prêts et validé par les équipes de risque. Nous avons aussi gardé un modèle linéaire parce qu'il reste explicable : l'article 22 du RGPD encadre les décisions entièrement automatisées, et une banque doit pouvoir dire à un client quels facteurs ont pesé dans son score. Le chiffrement ne change rien à cette exigence, puisque le modèle, lui, reste public.

Cette répartition entre poids publics et données secrètes est importante, car elle détermine le schéma nécessaire. Un produit « poids public × donnée chiffrée » et une somme de chiffrés suffisent : Paillier sait déjà le faire. Si les poids étaient eux aussi confidentiels, ou si le modèle contenait des interactions entre variables, il faudrait multiplier des chiffrés entre eux, donc passer à BFV ou CKKS.

Les vraies difficultés commencent avec ce que l'on fait du score. La sigmoïde n'est pas un polynôme, or un schéma homomorphe ne sait évaluer que des additions et des multiplications. On la remplace donc par une approximation polynomiale sur un intervalle donné, par exemple $\sigma(z) \approx 0{,}5 + 0{,}197\, z - 0{,}004\, z^3$, qui coûte deux multiplications supplémentaires et une perte de précision maîtrisée. La décision finale, « accorder si le score dépasse un seuil », est encore plus délicate : une comparaison, triviale en clair, est l'une des opérations les plus coûteuses en FHE arithmétique, car elle demande soit une approximation polynomiale de degré élevé, soit un passage par TFHE. Notre prototype fait donc un choix assumé : le serveur renvoie le score chiffré et c'est le client, seul détenteur de la clé, qui le déchiffre et applique la règle de décision.

4. État de l'art et écosystème

Sur le plan académique, la régression logistique sur données chiffrées est devenue un banc d'essai classique. La compétition iDASH, organisée dans le domaine de la génomique, a demandé dès 2017 d'entraîner une régression logistique sur des données chiffrées ; l'équipe de Kim, Song, Kim, Lee et Cheon l'a fait avec CKKS en utilisant justement une approximation polynomiale de la sigmoïde **[à vérifier : référence exacte, 2018]**. Dans la finance, IBM a mené avec une banque brésilienne un pilote de prédiction sur données chiffrées **[à vérifier : Banco Bradesco, 2019-2020]**. Les cas d'usage proches du nôtre sont la détection de fraude mutualisée entre établissements, où chaque banque veut profiter des données des autres sans les voir, et l'apprentissage fédéré, où le chiffrement homomorphe protège les mises à jour de modèle.

Côté outils, l'écosystème s'est beaucoup structuré :

- **Microsoft SEAL** (C++) implémente BFV, BGV et CKKS ; c'est la bibliothèque que nous utilisons à travers **TenSEAL**, son interface Python orientée tenseurs.
- **OpenFHE**, successeur de PALISADE, regroupe tous les schémas majeurs et le bootstrapping dans un même projet open source.
- **IBM HElib** a été l'une des premières bibliothèques complètes (BGV, puis CKKS).
- **Zama** propose TFHE-rs, le compilateur **Concrete** et **Concrete ML**, qui convertit un modèle scikit-learn en circuit FHE sans que le développeur ait à régler lui-même les paramètres.

La normalisation avance aussi. Le consortium HomomorphicEncryption.org a publié dès 2018 un standard qui fixe des jeux de paramètres garantissant 128 bits de sécurité, et des travaux sont en cours à l'ISO/IEC **[à vérifier : numéro de norme]**. Ce qui reste ouvert, en revanche, c'est la vitesse du bootstrapping, l'ergonomie pour des développeurs non spécialistes et la vérification de l'intégrité du calcul.

5. Le prototype : trois chiffrements pour un même score

Pour mesurer concrètement le prix de la confidentialité, nous avons construit une application web (FastAPI, servie dans un conteneur Docker) qui réalise le même calcul de score avec trois briques successives.

La première utilise **AES via Fernet**. Les données sont parfaitement protégées, mais le serveur ne peut rien en faire : il doit les déchiffrer pour calculer, ce qui le ramène au modèle actuel. La deuxième utilise **Paillier, que nous avons codé nous-mêmes** : le serveur reçoit trois chiffrés, élève chacun à la puissance de son poids, les multiplie et renvoie un chiffré du score. Les poids négatifs et les décimales sont gérés par un encodage en entiers mis à l'échelle. La troisième utilise **CKKS via TenSEAL** : le client génère ses clés, n'envoie au serveur qu'un contexte public (sans clé secrète) et un vecteur chiffré ; le serveur fait un produit scalaire chiffré avec le vecteur de poids. Pour rendre la séparation visible, la démo fait tenter au serveur un déchiffrement, qui échoue, preuve qu'il ne détient pas la clé.

Le protocole suivi est le même pour les deux briques homomorphes et tient en quatre temps. Le client génère sa paire de clés et chiffre ses trois indicateurs avec la clé publique. Il envoie au serveur les chiffrés et, pour CKKS, le contexte public qui contient les clés d'évaluation. Le serveur évalue le modèle, $\mathrm{Dec}(\mathrm{Eval}(f, \mathrm{Enc}(x_1), \ldots, \mathrm{Enc}(x_n))) = f(x_1, \ldots, x_n)$, sans jamais pouvoir lire ni les entrées ni le résultat. Le client, enfin, déchiffre le score et applique la règle de décision. L'interface affiche à chaque étape ce que voit chacun des deux acteurs, ce qui en fait un support de démonstration autant qu'un outil de mesure.

Le cas de Paillier a demandé une adaptation : le schéma ne connaît que des entiers positifs modulo n. Nous multiplions donc les valeurs et les poids par 1 000 avant de les arrondir, et un poids négatif est appliqué en élevant le chiffré à la puissance de l'inverse modulaire. Au déchiffrement, toute valeur supérieure à n/2 est interprétée comme négative, puis le résultat est divisé par 1 000 × 1 000. Ce petit encodage est précisément le genre de détail que les bibliothèques modernes cachent au développeur.

Les mesures reprises sur le poster ont été faites sur un même client (R = 3 200 €, E = 28 %, I = 1, score attendu 18,3), avec une clé Paillier de 2 048 bits et des paramètres CKKS de degré 8 192 (médianes de 10 à 20 exécutions, Linux x86_64, Python 3.13, TenSEAL 0.3.18) :

Mesure	En clair	Paillier 2 048 bits	CKKS (N = 8 192)
Calcul côté serveur	0,0005 ms	90 ms	4,4 ms
Chiffrement côté	—	260 ms	4,6 ms

Mesure	En clair	Paillier 2 048 bits	CKKS (N = 8 192)
client			
Déchiffrement	—	86 ms	1,1 ms
Données envoyées	24 octets	1,5 Ko	332 Ko
Erreur sur le score	—	nulle	6×10^{-6}

Trois enseignements ressortent. D'abord, le surcoût en temps est énorme en valeur relative, environ 8 000 fois le calcul en clair pour CKKS, mais faible en valeur absolue : quelques millisecondes par client, ce qui est parfaitement acceptable pour une décision d'octroi de crédit qui prend de toute façon des minutes ou des jours. Ensuite, le vrai coût est le volume : le vecteur chiffré CKKS pèse environ 14 000 fois les données brutes, et le contexte public, qui contient les clés de rotation nécessaires au produit scalaire, atteint environ 35 Mo, à transmettre une fois par client. Enfin, CKKS est approché par construction : l'erreur reste de l'ordre du millionième, négligeable pour un score, mais il faut l'accepter et la documenter, alors que Paillier donne un résultat exact.

Une observation nous a surpris : Paillier, conceptuellement plus simple, est ici plus lent que CKKS côté serveur. L'explication tient à l'arithmétique sur des entiers de 4 096 bits (modulo n^2) et au fait que CKKS traite tout le vecteur en une seule opération SIMD.

[À COMPLÉTER] Les difficultés rencontrées pendant le développement (installation de TenSEAL selon les systèmes, encodage des poids négatifs dans Paillier, choix des paramètres CKKS…) et ce que nous ferions autrement.

6. Transmettre : le cours et l'atelier fhe-cli

Le cours est déposé sur Moodle et suit la même progression que ce résumé. Un livre de cours va des rappels de cryptographie jusqu'à CKKS et TFHE, en passant par Paillier pas à pas, LWE, BGV/BFV et le bootstrapping. Une leçon interactive applique ces notions au scoring bancaire, avec des questions intermédiaires. Un second livre présente l'état de l'art, un troisième les limites, et une page ouvre sur les perspectives. Un glossaire d'environ 70 termes et un examen final de 40 questions à choix multiples complètent le parcours, avec des vidéos de cours et une vidéo de correction du cas pratique.

Pour le cas pratique, nous voulions que l'apprenant écrive du code plutôt que de le lire. Nous nous sommes inspirés du principe de rustlings, bien connu des développeurs Rust : une suite d'exercices courts, chacun avec un code à compléter et des tests. L'outil `fhe-cli` s'installe avec `pip install fhe-cli`. La commande `fhe-cli start` lance un atelier guidé qui relance les tests à chaque sauvegarde, et la solution de référence d'un exercice ne se débloque qu'une fois l'exercice réussi. Les dix exercices forment une progression : César, XOR et Fernet (exercices 1 à 3) ; exponentiation et inverse modulaires (4 et 5) ; clés et chiffrement Paillier (6 et 7) ; propriétés homomorphes et score chiffré (8 et 9) ; et un bonus en vrai FHE avec CKKS (10).

La diffusion a été un chantier en soi. Le paquet est publié automatiquement sur PyPI par une chaîne d'intégration continue qui exécute les tests sous Windows, macOS et Linux. Pour les postes sans Python, nous fournissons des exécutables autonomes qui embarquent l'interpréteur et TenSEAL, ce qui nous a obligés à composer avec la limite de 50 Mo par fichier imposée par Moodle. Une version web de l'atelier, lancée par Docker, propose les mêmes exercices dans le navigateur.

[À COMPLÉTER] Ce que les premiers retours d'utilisateurs (camarades, mentor) nous ont appris sur la difficulté des exercices.

7. Limites et regard critique

Le prototype montre que le FHE fonctionne ; il montre aussi ce qu'il ne règle pas.

Le coût. Pour un score linéaire, quelques millisecondes suffisent. Mais dès que le modèle s'approfondit (régression logistique avec sigmoïde approchée, arbres de décision, réseaux de neurones), la profondeur multiplicative augmente, les paramètres grossissent, et il faut recourir au bootstrapping. Les temps passent alors de la milliseconde à la seconde, voire davantage, par client.

La taille des chiffrés et des clés. Un facteur d'expansion de l'ordre de 10^4 et des clés d'évaluation de plusieurs dizaines de mégaoctets pèsent sur le réseau et le stockage, surtout à l'échelle de millions de clients.

L'expertise. Le choix du degré du polynôme, des modules, de l'échelle CKKS et du niveau de sécurité est un travail de spécialiste ; une erreur peut produire un résultat faux ou, pire, une sécurité insuffisante sans que rien ne le signale. Les compilateurs comme Concrete ML réduisent cette difficulté, sans la faire disparaître.

La gestion des clés. Dans notre scénario, c'est le client qui détient la clé secrète. Dans un système réel, il faut décider qui la génère, où elle est stockée, comment elle est révoquée, et comment la banque obtient finalement la décision dont elle a besoin.

L'intégrité. Le FHE garantit la confidentialité, pas l'exactitude du calcul : rien ne prouve au client que le serveur a appliqué le bon modèle. Il faudrait y ajouter des preuves de calcul vérifiable, encore coûteuses.

Le FHE n'est pas non plus la seule réponse. Le calcul multipartite sécurisé (MPC) répartit les données entre plusieurs serveurs qui ne doivent pas s'entendre ; il est souvent plus rapide en calcul mais très bavard en réseau. Les enclaves matérielles (TEE, comme Intel SGX ou AMD SEV) sont presque gratuites en performance, mais déplacent la confiance vers le fabricant du processeur et restent exposées aux attaques par canaux auxiliaires. La confidentialité différentielle, enfin, protège les statistiques publiées sur un ensemble de clients, pas la donnée individuelle utilisée pour un score. Le FHE est la seule de ces approches où la confiance repose uniquement sur une hypothèse mathématique ; c'est sa force, et ce qui justifie son coût.

La maturité pour un déploiement bancaire. Au-delà de la technique, une banque doit faire valider tout nouveau composant cryptographique par ses équipes de sécurité, ses auditeurs et parfois son régulateur. Les bibliothèques FHE sont jeunes à cette échelle de temps : leurs interfaces évoluent encore, peu de produits ont été certifiés, et les compétences sont rares sur le marché du travail. Le scénario le plus réaliste à court terme n'est donc pas de chiffrer tout le système de scoring, mais d'isoler un calcul précis à forte valeur de confidentialité, par exemple un score mutualisé entre plusieurs établissements, ou un calcul confié à un prestataire cloud, et d'y appliquer le FHE en complément des protections existantes.

8. Conclusion et perspectives

À la question posée au départ, notre réponse est nuancée. Pour un score linéaire ou une régression simple, le FHE est déjà exploitable : quelques millisecondes par client, une erreur négligeable, et un serveur qui ne voit jamais les données. Il reste pourtant réservé à des cas ciblés, là où la confidentialité justifie la taille des chiffrés, la complexité du paramétrage et l'effort d'intégration.

Trois évolutions peuvent changer la donne : les accélérateurs matériels (GPU, FPGA, puis circuits dédiés) qui visent un gain de plusieurs ordres de grandeur sur le bootstrapping ; les compilateurs qui masquent les paramètres au développeur ; et la normalisation, qui donnera aux banques et aux régulateurs un cadre de confiance commun.

[À COMPLÉTER] Bilan personnel de chacun : ce que le projet nous a apporté (cryptographie, pédagogie, publication d'un outil, travail en binôme) et la suite que nous aimerions lui donner.

Références

- P. Paillier, « Public-key cryptosystems based on composite degree residuosity classes », EUROCRYPT 1999.
- O. Regev, « On lattices, learning with errors, random linear codes, and cryptography », STOC 2005.
- C. Gentry, « Fully homomorphic encryption using ideal lattices », STOC 2009.
- V. Lyubashevsky, C. Peikert, O. Regev, « On ideal lattices and learning with errors over rings », EUROCRYPT 2010.
- Z. Brakerski, C. Gentry, V. Vaikuntanathan, « (Leveled) fully homomorphic encryption without bootstrapping », ITCS 2012.
- J. Fan, F. Vercauteren, « Somewhat practical fully homomorphic encryption », IACR ePrint 2012/144.
- J. H. Cheon, A. Kim, M. Kim, Y. Song, « Homomorphic encryption for arithmetic of approximate numbers », ASIACRYPT 2017.
- I. Chillotti, N. Gama, M. Georgieva, M. Izabachène, « TFHE: fast fully homomorphic encryption over the torus », Journal of Cryptology, 2020.
- C. Marcolla et al., « Survey on fully homomorphic encryption, theory, and applications », IACR ePrint 2022/1602.
- A. Al Badawi et al., « OpenFHE: open-source fully homomorphic encryption library », IACR ePrint 2022/915.
- A. Benaissa et al., « TenSEAL: a library for encrypted tensor operations using homomorphic encryption », 2021 ; Microsoft SEAL.
- M. Albrecht et al., « Homomorphic Encryption Standard », HomomorphicEncryption.org, 2018.